

「桌面應用程式開發」 網路應用程式

李東霖 博士

國立臺灣海洋大學 電機工程學系

2025 Fall semester

網路插座(Network socket)

- 網路插座(Network socket)在計算機科學中是電腦網路中行程間資料串流的端點。
- 在作業系統中，通常會為應用程式提供一組應用程式介面(Application Programming Interface, API)，稱為插座介面(socket API)。應用程式可以通過插座介面，來使用網路插座，以進行資料交換。
- 在網路通訊中socket常會與電腦的通訊埠(port)綁定。
 - 通常port的編號為0-65535
- 一個通訊埠在同一時間只能被一個程序使用！

通訊控制協定(TCP)

- TCP (Transmission Control Protocol)是位於OSI 參考模型中第四層傳輸層的一種網路通訊協定，主要目的是可確保資料通訊的正確傳輸以及提供流量控制。
- TCP是一種連線導向 (Connection Oriented)的傳輸協定。在封包傳送之前，必須先經過與對方建立連線之動作，當建立連線成功後，才能開始傳送資料給對方，等資料傳送完畢後，再關閉連線的一種傳輸方式。

使用者資料包協定(UDP)

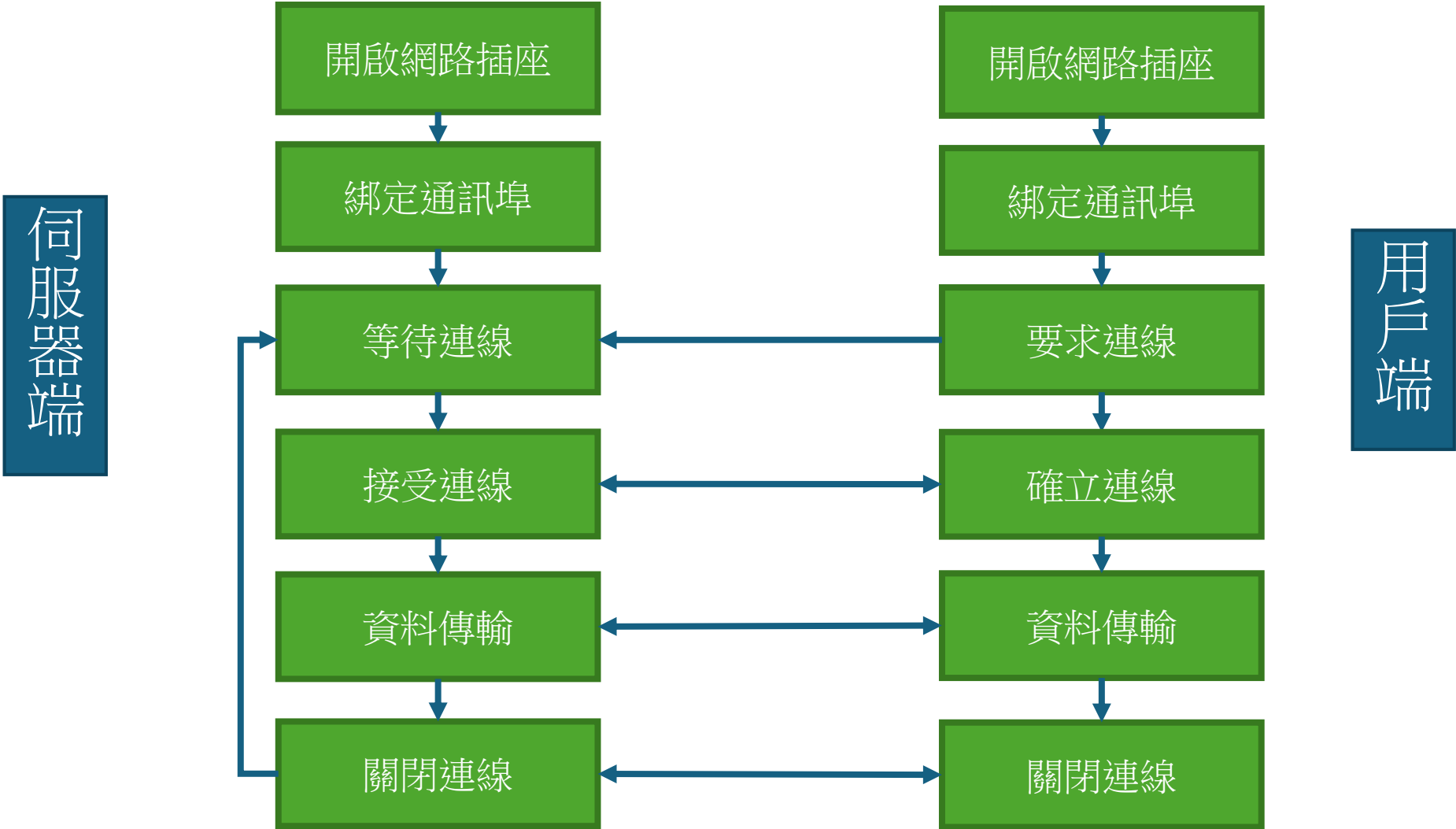
- UDP (User Datagram Protocol) 提供的是一個非可靠的非連線導向的資料流傳輸服務，也同樣是位於OSI 網路參考模型的第四層傳輸層中的一種網路通訊協定。
 - 非可靠表示不會對傳遞的內容進行順序確認與收到與否確認
 - 非連線表示傳送之前不會先通知傳送目標
- UDP與TCP相比少了很多確認機制，因此傳送速度較快。適用於許多即時服務。

TCP vs UDP 差異

項目	TCP	UDP
特性	連線導向	非連線導向
傳輸可靠度	高	低
適用	聊天室、檔案	影像串流、廣播

TCP實作

TCP連線建立流程



利用Socket建立基礎連線 (1)

- 載入

```
using System;  
using System.Net;  
using System.Net.Sockets;
```

- 建立一個監聽用Socket

```
static Socket socketListener;
```


建立Server端(1) - 利用Socket建立基礎連線

```
public static void Server(int myPort,int allowNum)
{
    //實作監聽用Socket，
    //AddressFamily.InterNetwork表示利用IP4協議
    //SocketType.Stream 因為我們需要使用TCP協議，需使用流式的Socket
    //ProtocolType.Tcp 選用TCP協議
    socketListener = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);
    //指定伺服器IP，這邊利用IPAddress.Any方法得到本機的IP，讓這方法可以運用更靈活
    IPAddress ip = IPAddress.Any;
    //設置應用程序的端口
    int port = myPort;
    //利用IPEndPoint這個類，將伺服器端的IP還有端口帶入給socketListener
    IPEndPoint point = new IPEndPoint(ip, port);
    //將point綁定給socketListener
    socketListener.Bind(point);

    ShowMsg("Listening..."); // 訊息顯示方式的函數(另建)
    //設定幾個程序可以連至本伺服器
    socketListener.Listen(allowNum);
}
```

建立Server端(2) - 訊息顯示

```
private static void ShowMsg(string s)
{
    Console.WriteLine(s); // 秀在Console
    // 也可以改秀在GUI元件上
}
```

建立Server端(3) - 傳送資料給客戶端

```
public static void ServerSender(Socket listener)
{
    //利用Accept方法接收監聽用Socket資料
    Socket socketSender = listener.Accept();
    //如果連線成功利用socketSender.RemoteEndPoint取得所連線到的Socket的IP和Port
    ShowMsg(("Client IP = " + socketSender.RemoteEndPoint.ToString()) + " Connect Succese!");
}
```

建立Server端(4) - 開始監聽

```
private void button1_Click (object sender, EventArgs e)
{
    //將端口設置為60000，設定3個應用程序可以連線到伺服器
    Server(60000,3);
    //取得客戶ServerSender(socketListener端資料
    );
}
```

建立Server端(5) - 利用telnet測試本程式

- 開啟Terminal並輸入

```
telnet 127.0.0.1 60000
```

Tip

Terminal中輸入

`lsof -i tcp` 顯示所有使用 T C P 協議的應用程序

`lsof -i udp` 顯示所有使用 U D P 協議的應用程序

`lsof -i tcp:port` 顯示該端口所使用的應用程序

建立可多人連線的Server端 (1)

- 新增2個載入

```
using System.Threading;  
using System.Text;
```

- 在啟動時使用下列方式

```
private void button1_Click (object sender, EventArgs e)  
{  
    Server(60000,1);  
    Thread th = new Thread(ServerSender);  
    th.Start(socketListener);  
}
```

建立可多人連線的Server端 (2)

- 修改監聽方式為持續監聽並增加客戶端資料處理執行緒

```
public static void ServerSender(object obListener)
{
    Socket temp = obListener as Socket;
    while (true)
    {
        Socket socketSender = temp.Accept();
        ShowMsg(("Client IP = " + socketSender.RemoteEndPoint.ToString()) + " Connect Succese!");
        //加開一個線程可重複接收客戶端所傳回的資料
        Thread th1 = new Thread(ReceiveClient);
        th1.IsBackground = true; //准許線程在背景執行
        Thread th2 = new Thread(SendMsgToClient);
        //啟動線程
        th2.Start(socketSender);
        th1.Start(socketSender);
    }
}
```

建立可多人連線的Server端 (3)

- 增加增加客戶端資料處理函數

```
static void ReceiveClient(object mySocketSender)
{
    Socket socketSender = mySocketSender as Socket;
    while (true)
    {
        //數據傳送是由數組的方式傳送
        //創立一個數組來儲存客戶端所回傳的訊息
        byte[] buffer = new byte[1024];
        //讀取字節數
        int rece = socketSender.Receive(buffer);
        //如果客戶端離開所得到的字節數會等於0，跳出此循環
        if (rece == 0)
        {
            ShowMsg(string.Format("Client : {0} + 下線了", socketSender.RemoteEndPoint.ToString()));
            break;
        }
    }
}
```

```
//載入System.Text的命名空間，利用GetString方法讀取字節，
//第一個引數代表要讀取的byte[]
//第二個引數代表從左邊數來第幾個字開始讀取
//每次讀取的字節數
string clientMsg = Encoding.UTF8.GetString(buffer, 0, rece);
ShowMsg(string.Format("Client : {0}", clientMsg));
```


建立可多人連線的Server端 (4)

- 新增回傳資料給客戶端的函數

```
private static void SendMsgToClient(Socket socketSender )
{
    while (true)
    {
        if (socketSender.RemoteEndPoint == null)
        {
            ShowMsg("socketSender.RemoteEndPoint == null");
            break;
        }
        string msg = "我收到你的訊息了"; //可自行調整由GUI元件輸入
        byte[] buffer = Encoding.UTF8.GetBytes(msg);
        socketSender.Send(buffer);
    }
}
```

建立Client端(1)

- 載入

```
using System;  
using System.Threading;  
using System.Net.Sockets;  
using System.Net;
```

- 建立一個連線用的socket

```
private static Socket socket;
```

建立Client端(2) - 利用Socket建立基礎連線

```
private static void ClientSocket(string ip, int port)
{
    socket = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);

    IPAddress myIp = IPAddress.Parse(ip);
    IPEndPoint point = new IPEndPoint(myIp, port);
    socket.Connect(point);

    ShowMsg("Connect Succese! "+socket.RemoteEndPoint.ToString());

    Thread sendMsg = new Thread(SendMsgToServer);
    Thread ReceiveMsg = new Thread(ReceiveMsgFromServer);

    ReceiveMsg.IsBackground = true;
    ReceiveMsg.Start();
    sendMsg.Start();
}
```

建立Client端(3) – 接收Server端訊息

```
private static void ReceiveMsgFromServer()
{
    while (true)
    {
        byte[] buffer = new byte[1024];

        int rec = socket.Receive(buffer);

        if (rec == 0)
        {
            ShowMsg("Server Loss!");
            break;
        }

        string receText = System.Text.Encoding.UTF8.GetString(buffer, 0, rec);
        ShowMsg("Server :" + receText);
    }
}
```

建立Client端(4) – 傳送訊息到Server端

```
private static void SendMsgToServer()
{
    while (true)
    {
        string inputText = textbox1.Text //可自行修改成其它輸入方式
        byte[] buffer = System.Text.Encoding.UTF8.GetBytes(inputText);
        socket.Send(buffer);
    }
}
```

建立Client端(4) – 訊息顯示方式

```
private static void ShowMsg(string s)
{
    Console.WriteLine(s); // 秀在Console
    // 也可以改秀在GUI元件上
}
```

建立Client端(5) – 連線Server

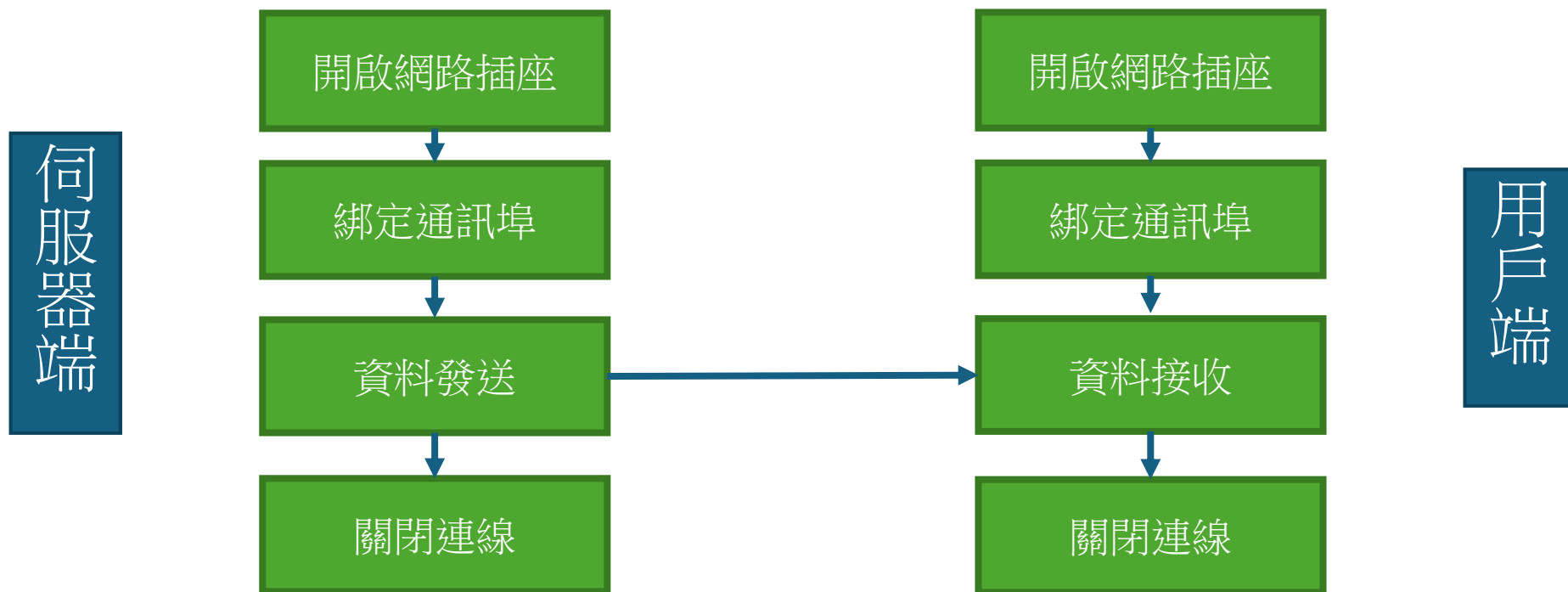
```
private void button1_Click (object sender, EventArgs e)
{
    string ip = "127.0.0.1"; //可自定義
    int port = 60000; //可自定義

    ClientSocket(ip,port);
}
```

測試

UDP實作

UDP連線建立流程



建立Server端 (1)

- 載入

```
using System;  
using System.Threading;  
using System.Net.Sockets;  
using System.Net;
```

- 建立Server發送資料端口

```
static Socket server;
```

建立Server端(2) – 開啟服務器

```
private void button1_Click (object sender, EventArgs e)
{
    server = new Socket(AddressFamily.InterNetwork, SocketType.Dgram, ProtocolType.Udp);
    server.Bind(new IPEndPoint(IPAddress.Parse("127.0.0.1"), 6001)); //綁定端口號和IP
    Console.WriteLine("服務端已經開啓"); //可自行調整秀至GUI元件上
    Thread t2 = new Thread(sendMsg); //開啓發送消息線程
    t2.Start();
}
```

建立Server端(3) – 傳送資料

```
static void sendMsg()
{
    EndPoint point = new IPEndPoint(IPAddress.Parse("127.0.0.1"), 6000);
    while (true)
    {
        string msg = "test";//自行調整
        server.SendTo(Encoding.UTF8.GetBytes(msg), point);
    }
}
```

建立Client端 (1)

- 載入

```
using System;  
using System.Threading;  
using System.Net.Sockets;  
using System.Net;
```

- 建立Server發送資料端口

```
static Socket client;
```

建立Client端(2) – 開啟客戶端

```
private void button1_Click (object sender, EventArgs e)
{
    server = new Socket(AddressFamily.InterNetwork, SocketType.Dgram, ProtocolType.Udp);
    server.Bind(new IPEndPoint(IPAddress.Parse("127.0.0.1"), 6000)); //綁定端口號和IP
    Console.WriteLine("服務端已經開啓"); //可自行調整秀至GUI元件上
    Thread t1 = new Thread(ReciveMsg); //開啓接收消息線程
    t1.Start();
}
```

建立Client端 (2) – 接收資料

```
static void ReciveMsg()
{
    while (true)
    {
        EndPoint point = new IPEndPoint(IPAddress.Any, 0); //用來保存發送方的ip和端口號
        byte[] buffer = new byte[1024];
        int length = server.ReceiveFrom(buffer, ref point); //接收數據報
        string message = Encoding.UTF8.GetString(buffer, 0, length);
        Console.WriteLine(point.ToString() + message); //可自行調整秀至GUI元件上
    }
}
```


測試